



SENATI



PLAN DE TRABAJO DEL ESTUDIANTE

RDA
RECURSO DIDÁCTICO PARA EL APRENDIZAJE

RECURSO DIDÁCTICO PARA EL APRENDIZAJE

DATOS DEL ESTUDIANTE

Apellidos y Nombres:	CALSIN MUÑOZ KEVIN JOHANER	ID:	1559805
Dirección Zonal/CFP:	Virtual		
Carrera:	Ingeniería De Software Con Inteligencia Artificial	Semestre:	IV
Curso/ Mód. Formativo:	FUNDAMENTOS Y ALGORITMIA PARA INTELIGENCIA ARTIFICIAL		
Tema de Trabajo Final:	ENTREGABLE 1 FINAL		

Respuestas a preguntas guía

Durante el análisis y estudio del caso práctico, debes obtener las respuestas a las interrogantes:

Pregunta 01:	Diferencia entre IA, Machine Learning, y Deep Learning
Inteligencia Artificial (IA): Es el campo de la informática que busca crear sistemas capaces de realizar tareas que normalmente requieren inteligencia humana, como el reconocimiento de voz, toma de decisiones y procesamiento del lenguaje natural. La IA incluye diversas técnicas, entre ellas Machine Learning y Deep Learning. Machine Learning (ML): Es un subconjunto de la IA que permite a los sistemas aprender automáticamente a partir de datos, sin ser explícitamente programados. Se basa en la construcción de algoritmos que identifican patrones en los datos y los utilizan para realizar predicciones o clasificaciones. Deep Learning (DL): Es una rama del ML que utiliza redes neuronales artificiales profundas. Estas redes tienen múltiples capas y son capaces de procesar grandes volúmenes de datos, extrayendo características de manera automática. DL es especialmente útil para tareas como reconocimiento de imágenes, procesamiento del lenguaje natural y conducción autónoma.	
Pregunta 02:	¿Cuál es la diferencia entre aprendizaje supervisado, no supervisado y por reforzamiento? Da un ejemplo de cada uno.
Aprendizaje Supervisado: En este tipo de aprendizaje, el modelo se entrena con datos etiquetados, lo que significa que cada entrada tiene una respuesta correcta conocida. El modelo aprende a mapear los datos de entrada con la salida deseada. Ejemplo: Un modelo de clasificación de correos electrónicos que aprende a distinguir entre spam y correos legítimos con datos etiquetados. Aprendizaje No Supervisado: Aquí el modelo trabaja con datos sin etiquetas y busca estructuras y patrones en los datos. Su objetivo es descubrir relaciones ocultas sin intervención humana. Ejemplo: Segmentación de clientes en marketing, donde los algoritmos agrupan consumidores con características similares sin conocer etiquetas previas. Aprendizaje por Reforzamiento: En este enfoque, el modelo aprende mediante prueba y error en un entorno interactivo. Se le proporcionan recompensas o castigos según sus acciones, con el objetivo de maximizar su desempeño a largo plazo. Ejemplo: Un agente de IA que juega ajedrez aprende estrategias óptimas al recibir recompensas por movimientos exitosos y penalizaciones por jugadas ineficaces.	

Pregunta 03:	¿Qué papel juegan los conjuntos de datos (datasets) en el rendimiento de un modelo de ML?
Los conjuntos de datos son fundamentales para el desempeño de los modelos de ML. Algunos aspectos clave incluyen: • Calidad de los datos: Datos limpios, sin ruido ni errores, garantizan predicciones más precisas. • Cantidad de datos: Un mayor volumen de datos puede mejorar la generalización del modelo y evitar el sobreajuste. • Diversidad de los datos: Datos representativos de diversas condiciones ayudan a crear modelos más robustos. • Preprocesamiento: Técnicas como normalización, eliminación de valores atípicos y llenado de valores faltantes mejoran el rendimiento del modelo.	
Pregunta 04:	¿Cuáles son los principales algoritmos usados en aprendizaje supervisado, y en qué casos se suele escoger cada uno?
• Regresión Lineal: Se usa para problemas de predicción de valores continuos, como el precio de viviendas basado en características como tamaño y ubicación. • Regresión Logística: Ideal para problemas de clasificación binaria, como detección de enfermedades (positivo o negativo). • Árboles de Decisión: Útiles en problemas donde la interpretabilidad es clave, como diagnóstico médico basado en síntomas. • Máquinas de Soporte Vectorial (SVM): Se utilizan en clasificación de alto rendimiento cuando los datos tienen una estructura clara. • Redes Neuronales Artificiales: Son preferidas para tareas complejas como reconocimiento de voz o visión artificial, donde los patrones no son fácilmente identificables con modelos tradicionales. • K-Nearest Neighbors (KNN): Funciona bien en problemas de clasificación cuando hay relaciones cercanas entre los puntos de datos.	

1. PLANIFICACIÓN DEL TRABAJO

▪ Cronograma de actividades:

N°	ACTIVIDADES	CRONOGRAMA					
		25/04	28/04	29/04	30/04	02/05	
	Elaboración del informe (Formato del alumno)	X					
	Elaboración del código		x				
	Elaboración del ppt			x			
	Subir informe y el script				x		
	exposición				x	x	

▪ Lista de recursos necesarios:

1. MÁQUINAS Y EQUIPOS	
Descripción	Cantidad
COMPUTADORA	1
CELULAR	1
DISCO DURO	1

2. HERRAMIENTAS E INSTRUMENTOS	
Descripción	Cantidad
EDITOR DE CODIGO (VS CODE)	1
REPOSITORIO GIT HUB	1

3. MATERIALES E INSUMOS	
Descripción	Cantidad
PPTT DE INFORME	1

2. DECIDIR PROPUESTA

- Describe la propuesta determinada para la solución del caso práctico

PROPUESTA DE SOLUCIÓN

La solución propuesta combina conceptos teóricos y prácticos de Inteligencia Artificial, Machine Learning y Deep Learning, junto con la manipulación de matrices y operaciones numéricas en Python utilizando NumPy. El proyecto tiene dos componentes principales:

1. Parte Teórica (Presentación)
 - Se desarrolló una presentación donde se explican las diferencias entre IA, Machine Learning y Deep Learning, así como los distintos tipos de aprendizaje (supervisado, no supervisado y por refuerzo).
 - Además, se abordó la importancia de los datasets en el rendimiento de los modelos de aprendizaje automático y una descripción de los principales algoritmos usados en aprendizaje supervisado.
2. Parte Práctica (Código en Python)
 - Se diseñó un script en Python que permite generar matrices y realizar pruebas con propiedades aritméticas fundamentales (conmutativa, asociativa, distributiva, inverso aditivo e identidad).
 - El programa solicita datos directamente al usuario, validando que los valores ingresados sean correctos y asegurando la integridad de los datos para evitar errores.
 - En la ejecución, el código demuestra que las propiedades matemáticas se cumplen, proporcionando una visión clara de cómo se aplican estos principios en la computación y el análisis de datos.



3. EJECUTAR

- Resolver el caso práctico, utilizando como referencia el problema propuesto y las preguntas guía proporcionadas para orientar el desarrollo.
- Fundamentar sus propuestas en los conocimientos adquiridos a lo largo del curso, aplicando lo aprendido en las tareas y operaciones descritas en los contenidos curriculares.

INSTRUCCIONES: Ser lo más explícito posible. Los gráficos ayudan a transmitir mejor las ideas. Tomar en cuenta los aspectos de calidad, medio ambiente y SHI.

```

entregable.py> ...
1 import numpy as np
2
3 # Función para pedir al usuario un vector validado (solo números enteros)
4 def obtener_vector(nombre):
5     while True:
6         try:
7             valores = input(f"Ingrese los valores del vector {nombre} (solo números enteros) separados por espacios: ")
8             vector = np.array([int(x) for x in valores.split()]) # Convertir a enteros
9             if vector.size == 0:
10                 raise ValueError("El vector no puede estar vacío.")
11             return vector
12         except ValueError:
13             print("Error: Asegúrese de ingresar SOLO números enteros separados por espacios. Intente nuevamente.")
14
15 # Función para pedir las dimensiones de una matriz validada
16 def obtener_dimensiones_matriz():
17     while True:
18         try:
19             dimensiones = input("Ingrese el número de filas y columnas de la matriz (ejemplo: 3 3): ")
20             filas, columnas = [int(x) for x in dimensiones.split()]
21             if filas <= 0 or columnas <= 0:
22                 raise ValueError("Las dimensiones deben ser mayores que cero.")
23             return filas, columnas
24         except ValueError:
25             print("Error: Ingrese dos números enteros positivos separados por un espacio. Intente nuevamente.")
26
27 # Generar matriz aleatoria con dimensiones dadas por el usuario
28 def generar_matriz():
29     filas, columnas = obtener_dimensiones_matriz()
30     matriz = np.random.randint(1, 10, (filas, columnas))
31     print("\nMatriz generada:")
32     print(matriz)
33     return matriz
34
35 # Propiedad conmutativa (A + B = B + A)
36 def conmutativa_suma(vector1, vector2):
37     return np.array_equal(vector1 + vector2, vector2 + vector1)
38
39 # Propiedad asociativa ((A + B) + C = A + (B + C))
40 def asociativa_suma(vector1, vector2, vector3):
41     return np.array_equal((vector1 + vector2) + vector3, vector1 + (vector2 + vector3))
42

```

```

entregable.py> ...
35 # Propiedad conmutativa (A + B = B + A)
36 def conmutativa_suma(vector1, vector2):
37     return np.array_equal(vector1 + vector2, vector2 + vector1)
38
39 # Propiedad asociativa ((A + B) + C = A + (B + C))
40 def asociativa_suma(vector1, vector2, vector3):
41     return np.array_equal((vector1 + vector2) + vector3, vector1 + (vector2 + vector3))
42
43 # Propiedad distributiva (A * (B + C) = A * B + A * C)
44 def distributiva(vector1, vector2, vector3):
45     return np.array_equal(vector1 * (vector2 + vector3), (vector1 * vector2) + (vector1 * vector3))
46
47 # Inverso aditivo (A + (-A) = 0)
48 def inverso_aditivo(vector):
49     return np.array_equal(vector + (-vector), np.zeros_like(vector))
50
51 # Identidad (A * I = A)
52 def identidad(vector):
53     identidad_matriz = np.eye(len(vector))
54     return np.array_equal(vector @ identidad_matriz, vector)
55
56 # Solicitar valores al usuario
57 vector_a = obtener_vector("A")
58 vector_b = obtener_vector("B")
59 vector_c = obtener_vector("C")
60 matriz = generar_matriz()
61
62 # Mostrar resultados
63 print("\nResultados de las propiedades aritméticas:")
64 print(f"Commutativa de la suma: {conmutativa_suma(vector_a, vector_b)}")
65 print(f"Asociativa de la suma: {asociativa_suma(vector_a, vector_b, vector_c)}")
66 print(f"Distributiva: {distributiva(vector_a, vector_b, vector_c)}")
67 print(f"Inverso aditivo: {inverso_aditivo(vector_a)}")
68 print(f"Identidad: {identidad(vector_a)}")
69

```


DIBUJO / ESQUEMA / DIAGRAMA DE PROPUESTA

(Adicionar las páginas que sean necesarias)

```

PS D:\senati\fundamentos de algoritmia\entregablefinal1> & C:/Users/Pc/AppData/Local/Programs/Python/Python313/python.exe "d:/senati/fundamentos de algoritmia/entregablefinal1/entregable.py"
Ingrese los valores del vector A (solo números enteros) separados por espacios: 3
Ingrese los valores del vector B (solo números enteros) separados por espacios: 5
Ingrese los valores del vector C (solo números enteros) separados por espacios: 1
Ingrese el número de filas y columnas de la matriz (ejemplo: 3 3): 4 3

Matriz generada:
[[4 4 6]
 [2 4 3]
 [2 4 3]
 [5 5 9]]

Resultados de las propiedades aritméticas:
Conmutativa de la suma: True
Asociativa de la suma: True
Distributiva: True
Inverso aditivo: True
Identidad: True
PS D:\senati\fundamentos de algoritmia\entregablefinal1>

```

 SENATI	[NOMBRE DEL TEMA DEL TRABAJO FINAL]	
	[APELLIDOS Y NOMBRES]	[ESCALA]

4. CONTROLAR

- Verificar el cumplimiento de los procesos desarrollados en la propuesta de solución del caso práctico.

EVIDENCIAS	CUMPLE	NO CUMPLE
• ¿Se identificó claramente la problemática del caso práctico?	☒	☐
• ¿Se desarrolló las condiciones de los requerimientos solicitados?	☒	☐
• ¿Se formularon respuestas claras y fundamentadas a todas las preguntas guía?	☒	☐
• ¿Se elaboró un cronograma claro de actividades a ejecutar?	☒	☐
• ¿Se identificaron y listaron los recursos (máquinas, equipos, herramientas, materiales) necesarios para ejecutar la propuesta?	☒	☐
• ¿Se ejecutó la propuesta de acuerdo con la planificación y cronograma establecidos?	☒	☐
• ¿Se describieron todas las operaciones y pasos seguidos para garantizar la correcta ejecución?	☒	☐
• ¿La propuesta es pertinente con los requerimientos solicitados?	☒	☐
• ¿Se evaluó la viabilidad de la propuesta para un contexto real?	☒	☐

5. VALORAR

- Califica el impacto que representa la propuesta de solución ante la situación planteada en el caso práctico.

CRITERIO DE EVALUACIÓN	DESCRIPCIÓN DEL CRITERIO	PUNTUACIÓN MÁXIMA	PUNTAJE CALIFICADO POR EL ESTUDIANTE
Identificación del problema	Claridad en la identificación del problema planteado.	3	
Relevancia de la propuesta de solución	La propuesta responde adecuadamente al problema planteado y es relevante para el contexto del caso práctico.	8	
Viabilidad técnica	La solución es técnicamente factible, tomando en cuenta los recursos y conocimientos disponibles.	6	
Cumplimiento de Normas	La solución cumple con todas las normas técnicas de seguridad, higiene y medio ambiente.	3	
PUNTAJE TOTAL		20	

